

CS566 Homework4

Due – Apr 2, 6:00 PM, at the class beginning

Student's Name: _____

All work must be your own. Upload copy of your homework and any supporting materials (source code and test cases) online to Assignment4 in the CS566_A2 course site
Show your work in detail.

Q1 [10 pts.]: Given the following key sequence (6,22,9,14,13,1,8) build the dynamic binary search tree WITHOUT BALANCING IT. How many probes (i.e., comparisons) does it take to determine that key 100 is not in the tree?

To figure out how many probes it takes to check if the key 100 is in a dynamic binary search tree built from the sequence (6, 22, 9, 14, 13, 1, 8), we start by building the tree with those keys. We start with 6 as the root, then add 22 as the right child. Next, we insert 9 as the left child of 22, followed by 14 as the right child of 9. We add 13 as the left child of 14, then put 1 as the left child of 6, and finally, we add 8 as the right child of 9. So the final structure has 6 at the top, 1 to the left, and 22 to the right. Then, 22 has 9 on the left, which has 8 on the right and 14 on the right, with 13 as its left child.

Now, for searching the key 100, we start at the root (6) and compare it to 100. Since 100 is greater than 6, that's one probe. Next, we go to the right child (22) and check again. Since 100 is greater than 22, that's another probe. Then, we look for a right child of 22 but find that it's null, meaning there are no more nodes to check. Since we hit a null pointer, we can conclude that 100 isn't in the tree. Overall, it takes **3 probes** to figure out that the key 100 is not in the binary search tree.

Q2 [15 pts.]: Find the depth-first search tree for the graph Fig.1 with G as the starting vertex. Assume that each adjacency list is in alphabetical order.

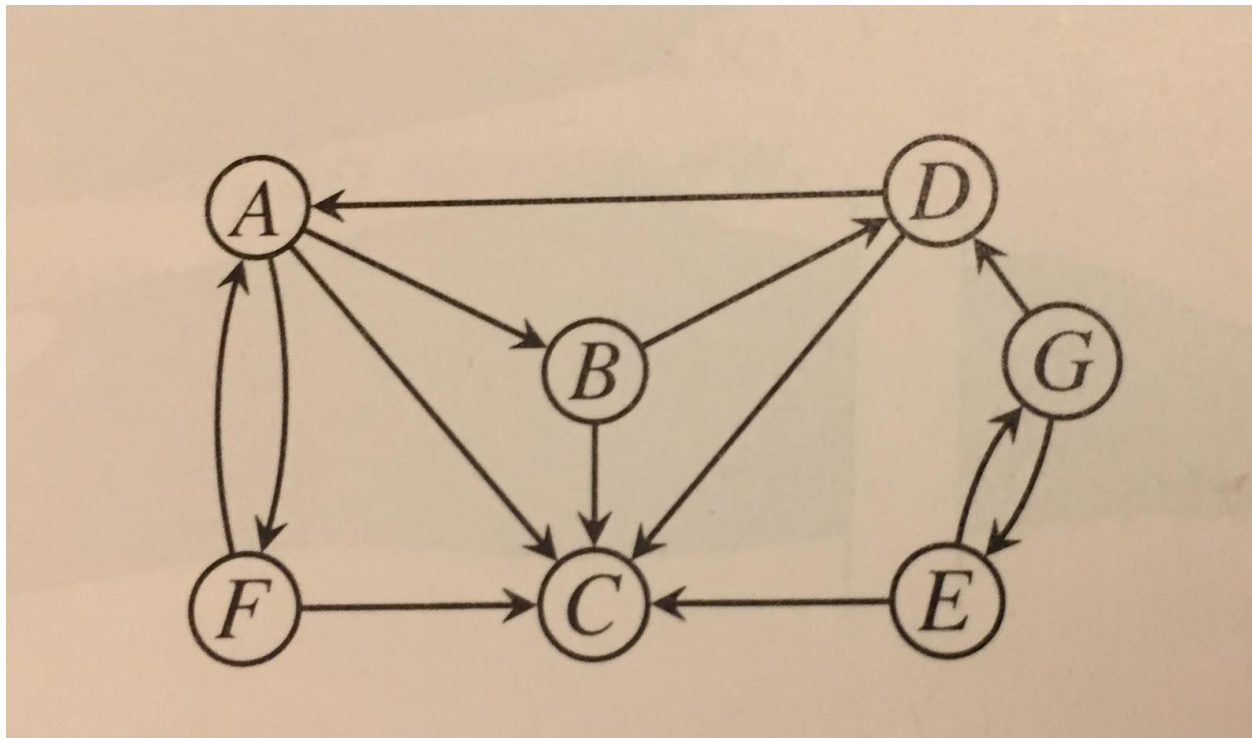


Fig.1

To solve the depth-first search (DFS) tree for the directed graph in Fig.1, starting from vertex G and following alphabetical order for each adjacency list, we begin at G. The first neighbor of G alphabetically is D, so we go there. From D, the first unvisited neighbor is A. From A, we move to B, and from B, we head to C. C has no outgoing edges that lead to any unvisited nodes, so we backtrack. We return to B, but it's fully explored. Then back to A, which still has F as an unvisited neighbor, so we continue to F. F only points back to A, which we've already visited, so we backtrack again to A, then to D. D has another neighbor, C, but C has already been visited through B, so we skip it. Now back at G, we check the next neighbor in alphabetical order, which is E. E's only neighbor is G, but G is the starting node and has already been visited, so we don't revisit it.

The final DFS tree starts at G, goes to D, then to A, from there to B, then to C, then back to A to explore F, and finally from G to E. So the structure of the DFS tree would look like:

G -> D -> A -> B -> C, with A -> F, and lastly G -> E.

This represents the path the search takes while always choosing the next vertex in alphabetical order and going as deep as possible before backtracking.

So, the DFS tree we end up with starts from G and branches out to E and D. From D, we go to A, which then branches out to B, leading to C, and also to F. This structure shows how we navigated through the graph, with G at the top as the root and E and D hanging off it.

Q3 [15 pts.]: Find the breadth-first search tree for the graph Fig.1 with G as the starting vertex. Assume that each adjacency list is in reverse alphabetical order

We begin by visiting G and queuing its neighbors. In reverse alphabetical order, that means we visit D first and then E. So, the first level of our BFS tree branches out from G to D and E.

Next, we explore D. Its neighbors in reverse order are C and A. Since neither has been visited yet, we add both C and A to our tree as children of D. Then we move on to E, but its only neighbor is G, which we've already visited, so we don't add anything new from E. Now we look at C, which doesn't have any outgoing edges, so we move on to A.

At A, its neighbors in reverse order are F, C, and B. C has already been visited, so we skip it and add F and B as new children of A. Finally, we check B, whose neighbors in reverse order are D and C, both of which have already been visited, so there's nothing else to add. F only connects to A, which has also already been visited, so the search ends there.

So, in the final BFS tree starting from G, the structure is: G connects to D and E; D connects to C and A; and A connects to F and B.

Q4 [20 pts.]: Execute Prim's minimum spanning tree algorithm by hand on the graph in Fig.2, showing how the data structures evolve. Clearly indicate which edges become part of minimum spanning tree and in what order. Start at vertex H.

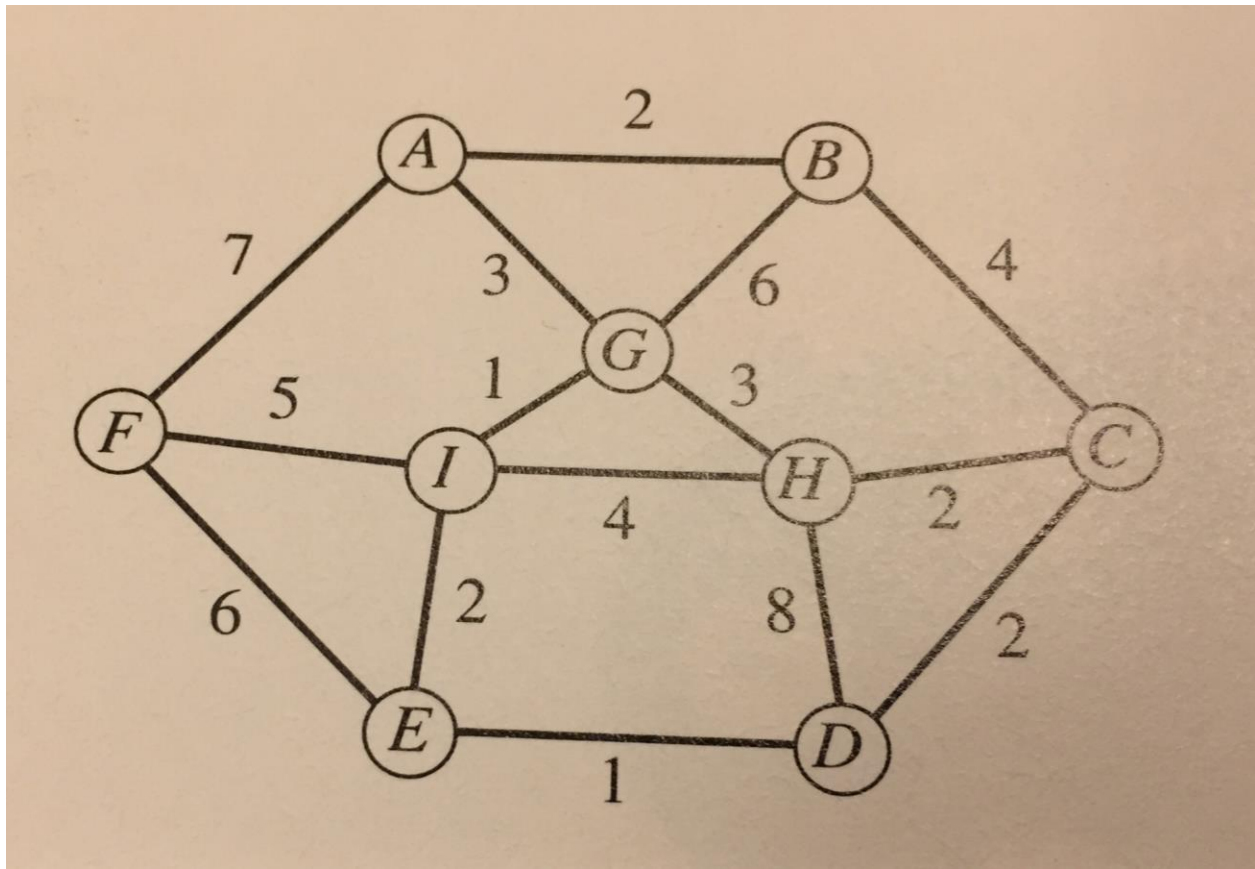


Fig.2

I first looked at all the edges connected to H: H-C (2), H-D (8), H-G (3), and H-I (4). The smallest one is H-C with weight 2, so I added that to the tree and brought C in. From the current tree (H and C), I looked at all connected edges: C-D (2), C-B (4), and the remaining H edges. The next smallest edge was C-D (2), so I added that and included D. Now with H, C, and D in the tree, I picked the smallest edge out of the new options, which was D-E with weight 1, so I added that and included E. From there, E-I (2) was the lowest edge available, so I brought in I. Next, I looked back and saw H-G was still on the table with a weight of 3, which was the smallest now, so I added that and included G.

At this point, I had H, C, D, E, I, and G in the tree. I kept going and found the edge G-A with weight 3, added that, and brought in A. Now the edge A-B (2) was the smallest among the remaining options, so I picked that next and added B to the tree. Finally, the only vertex left out was F, and the cheapest way to get to it was through I-F, which had a weight of 5. I added that last edge and completed the tree.

In total, I ended up including the edges: H-C (2), C-D (2), D-E (1), E-I (2), H-G (3), G-A (3), A-B (2), and I-F (5), which connected all vertices A through I while keeping the total weight at a minimum.

Q5 [20 pts.]: Dijkstra's shortest path algorithm.

Here are the adjacency list with edge weights in parentheses for a digraph Fig.3

A: B (4.0), F (2.0)

B: A (1.0), C (3.0), D (4.0)

C: A (6.0), B(3.0), D(7.0)

D: A (6.0), E(2.0)

E: D (5.0)

F: D(2.0), E(3.0)

Execute Dijkstra's shortest-path algorithm by hand on this graph, showing how the data structures evolve, with $s = A$. Clearly indicate which edge become part of the shortest – path tree in what order.

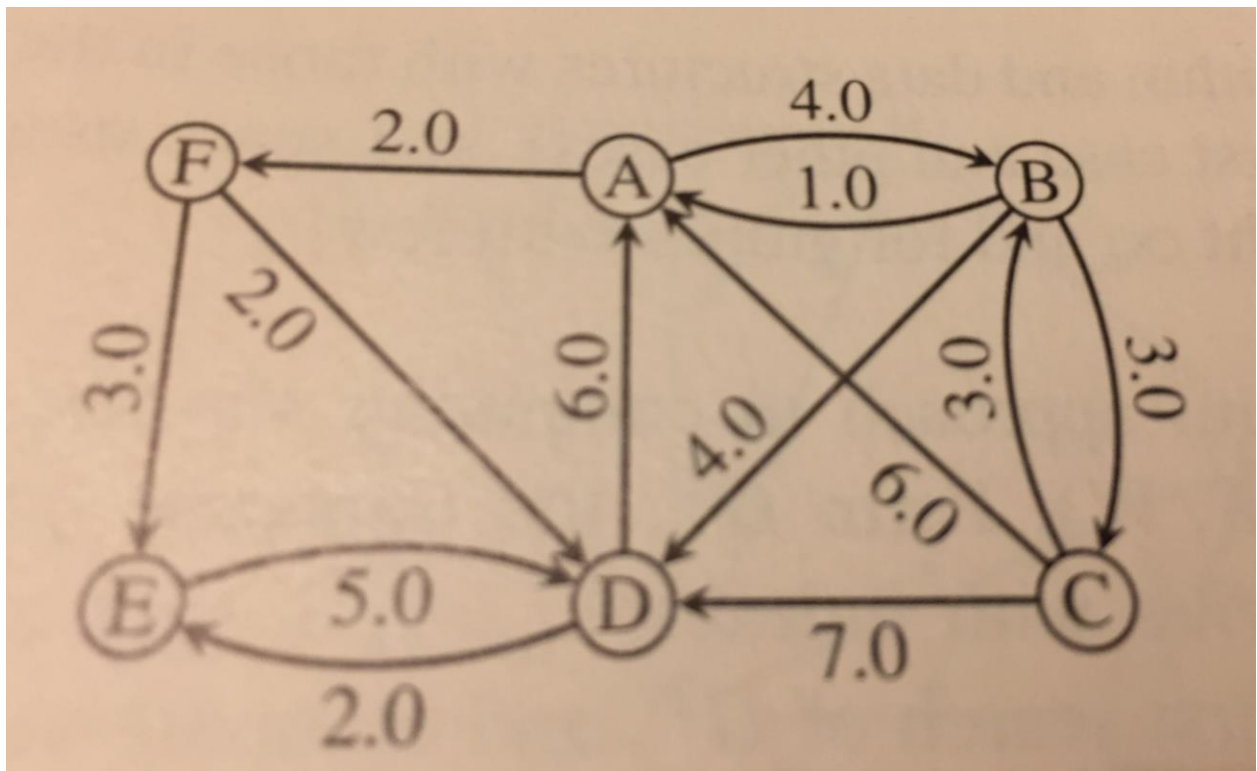


Fig.3

we first set the distance to A as 0 and all other vertices to infinity. From A, we look at the edges connected to it: A to B (weight 4.0) and A to F (weight 2.0). The smallest edge here is A to F, so we add that edge to our shortest-path tree and update the distance to F to 2, with A as its predecessor. Next, we check the edges from F, which connect to D (weight 2.0) and E (weight 3.0). We update the distance to D to 4 ($2 + 2$) and to E to 5 ($2 + 3$), making F the predecessor for D.

Now, we move to the next unvisited vertex with the smallest distance, which is B (distance 4.0). We check the edges from B to C (weight 3.0) and back to A (which we skip since it's already visited). The distance to C becomes 7 ($4 + 3$), but D remains at 4 since it's already been reached via F. After processing B, we go to D next, but it doesn't provide any new paths since its edges have already been covered. Then we look at E, which connects back to D (already visited) and doesn't change any distances.

Finally, we visit vertex C, but since its connections are to A and B (both already processed), no updates happen. At the end of this process, we've got the shortest paths from A to all other vertices. The final paths are A to B (4.0), A to F (2.0), A to D (4.0 via F), A to E (5.0 via F), and A to C (7.0 via B). The edges that make up the shortest-path tree, in the order they were added, are A to F (2.0), A to B (4.0), F to D (2.0), and F to E (3.0), along with B to C (3.0).

Q6 [20 pts.]: [Kruskal's minimum spanning tree algorithm]

Find the minimum spanning tree for the graph in Fig.4 that would be the output by Kruskal's algorithm, assuming that edges are sorted as shown on Fig.4

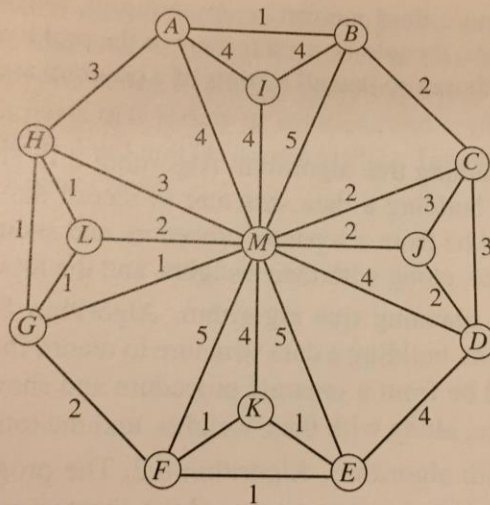


Figure 8.14 Sorted edges: AB, EF, EK, FK, GH, GL, GM, HL, BC, CM, DJ, FG, JM, LM, AH, CD, CJ, HM, AI, AM, BI, DE, DM, IM, KM, BM, EM, FM.

Fig.4

The first edge, AB with weight 1, was added right away because the MST was empty. Next, I added EF (1) and EK (1) since they connected new vertices and didn't cause any cycles. Then I looked at FK (1), but I had already connected E, F, and K through EF and EK, so adding FK would create a cycle so I skipped it.

Moving on, I added GH (1) because G and H were still separated from the rest. Then GL (1) came in cleanly, and GM (1) after that since they all kept connecting new vertices without any loops. HL (1) would've made a cycle through G and H, so I skipped it. After that, I added BC (2), which extended the MST, and then CM (2), which was also a safe addition. At this point, I realized I had already connected all the vertices in the graph, so I stopped going through the list since adding more edges would just create cycles.

So, the final MST includes the edges: AB (1), EF (1), EK (1), GH (1), GL (1), GM (1), BC (2), and CM (2). That's 8 edges for 9 vertices, which is exactly what you need for a valid MST. I just kept it simple: followed the edge list, watched out for cycles, and stopped once everything was connected.